

Structures, Classes and Object-Oriented Foundations in C++

[DEF] Definition to Remember

A struct in C++ is a user-defined type identical to a class except that its members and base classes are public by default. Structures encapsulate data attributes and member functions.

1. Concept Overview & Fundamentals

Structures, Classes and Object-Oriented Foundations in C++ provides modern high-performance capabilities combining low-level hardware control with high-level abstractions, object-oriented paradigms, and Standard Template Library (STL) utilities.

In modern C++ (C++11 through C++20):

* Ze

2. Technical Architecture & Memory Mechanics

C++ Runtime Execution Architecture



Modern C++ Feature Comparison

Modern C++ Paradigm	Legacy Equivalent	Engineering Advantage	Performance Impact
<code>**std::cin` / `std::cout`**</code>	<code>`scanf` / `printf`</code>	Type-safe, extensible for custom class	Buffered I/O (optimizable via <code>`ios::sy</code>
<code>**References (`Type&`)**</code>	Pointers (<code>`Type*`</code>)	Non-nullable, clean syntax without der	Direct alias; zero runtime overhead
<code>**std::vector<T>`**</code>	Raw Dynamic Array (<code>`malloc`</code>)	Automatic resizing, bounds safety, mem	Amortized <code>**O(1)**</code> push_back, contiguo
<code>**`nullptr`**</code>	<code>`NULL` / `0`</code>	Strongly typed pointer literal prevent	Zero overhead

3. Core Syntax & Implementation Patterns

Modern C++ Template Structure

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

// Fast I/O and standard namespace configuration
int main() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    std::cout << "Executing C++ Module: Structures" << "\n";

    return 0;
}
```

4. Real-World Applications & System Engineering

- * Quantitative Finance & HFT: High-frequency trading platforms use modern C++ for microsecond-level execution.

* Ga

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: book array initializer

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Ini
 */

/*
 * ===
 * COM
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ===
 */

#include
#include
#include
```

```
// Use
using

struct
    st
    st
    do
};

//
int ma
//
ve

};

//
cc
//
fo

}

//
re
}
```

Problem 2: book structure

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Cre
demonst
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 * ==
```

```
*/  
  
#inclu  
#inclu  
  
// Use  
using  
  
struct  
    st  
    st  
    do  
};  
  
void c  
    //  
    co  
    //  
    co  
    //  
    co  
}  
  
    //  
int ma  
    Bo  
    b1  
    b1  
    b1  
    b1  
  
    //  
    co  
    di  
  
    //  
    re  
}
```

Problem 3: car description function

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*  
 * Que  
 * Pas  
 */  
  
/*  
 * ===  
 * CON  
 * ===  
 * 1.  
 *  
 *  
 * 2.  
 *  
 *  
 * 3.  
 *  
 *  
 */
```

```
* 4.
* *
*
*
*
* ===
*/

#include
#include

// Use
using

struct
    st
    st
    in
    st
};

void p
    //
    co

}

//
int ma
    Ca
    pr
    //
    re
}
```

Problem 4: car structure

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Def
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
```

```
* *
*
*
*
* ===
*/

#include
#include

// Use
using

struct
    st
    st
    in
    st
};

//
int ma
    Ca
    ca
    ca
    ca
    ca
    ca

//
cc
//
cc

//
re
}
```

Problem 5: nested structure student books

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Wr.
 * usage.
 */

/*
 * ===
 * CON
 * ===
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
```

```
*
* 4.
* *
*
*
*
* ===
*/

#include
#include
#include

// Use
using

struct
    st
    st
};

struct
    in
    st
    //
    ve
};

void c
    //
    co
    //
    co
    //
    fo

}
}

//
int ma
    St
    st
    st
    st

};

di

//
re

}
```

Problem 6: student modify gpa

Problem Description & Objective:

C++ Code Implementation:

Implem

```
/*
 * Que
 * Wri
grades
 */

/*
 * ==
 * CON
 * ==
 * 1.
 *
 *
 * 2.
 *
 *
 * 3.
 *
 *
 * 4.
 * *
 *
 *
 *
 * ==
 */

#include
#include

// Use
using

struct
    in
    st
    in
    do
};

void u
    //
    if
    }
}

//
int ma
St

//
co

up

//
co

//
re
}
```


6. Exam Scoring Points & Memory Revision

[MUST-WRITE] Must-Write Points

1. struct vs class in C++: struct defaults to public access; class defaults to private access.
2. Structures in C++ can have constructors, destructors, and member methods.
3. Array of structures: Efficiently stores and processes database records (e.g. Students, Employees).
4. Passing structs: Prefer pass-by-const-reference (const StructType&) to avoid full object copying.
5. Memory layout: Total size = sum of member sizes + alignment padding bytes.

[RECALL] Quick Recall

Encapsulated Attributes -> Member Functions -> Constructors -> public default -> const Ref Passing